

```

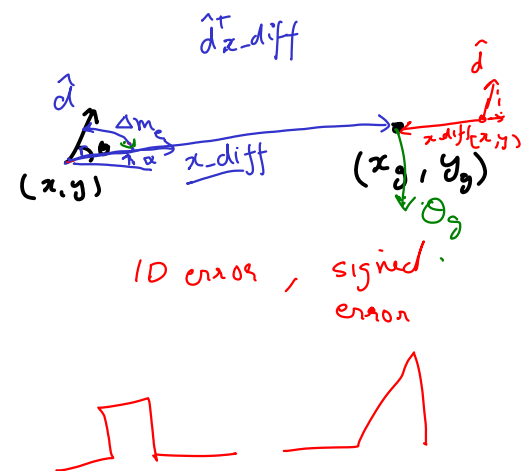
1 """
2 Taken from: https://github.com/AtsushiSakai/PythonRobotics/blob/master/Control/move\_to\_pose
/move_to_pose.py
3
4 Move to specified pose
5
6 Author: Daniel Ingram (daniel-s-ingram)
7         Atsushi Sakai (@Atsushi_twi)
8         Seied Muhammad Yazdian (@Muhammad-Yazdian)
9
10 P. I. Corke, "Robotics, Vision & Control", Springer 2017, ISBN 978-3-319-54413-7
11
12 """
13
14 import matplotlib.pyplot as plt
15 import numpy as np
16 from random import random
17
18 from ilqr import iLQRController
19
20 class Angle:
21     @staticmethod
22     def wrap(theta):
23         return ((theta + np.pi) % (2*np.pi)) - np.pi
24
25     @staticmethod
26     def iswrapped(theta):
27         return (-np.pi <= theta) & (theta < np.pi)
28
29     @staticmethod
30     def diff(a, b):
31         assert Angle.iswrapped(a).all()
32         assert Angle.iswrapped(b).all()
33         # np.where is like a conditional statement in numpy
34         # but it operates on per element level inside the numpy array
35         return np.where(a < b,
36                         (2*np.pi + a - b),
37                         (a - b))
38
39     @staticmethod
40     def dist(a, b):
41         # The distance between two angles is minimum of a - b and b - a.
42         return np.minimum(Angle.diff(a, b), Angle.diff(b, a))
43
44 class PIDController:
45     """
46     Constructs an instantiate of the PIDController for navigating a
47     3-DOF wheeled robot on a 2D plane
48
49     Parameters
50     -----
51     Kp_rho : The linear velocity gain to translate the robot along a line
52              towards the goal
53     Kp_alpha : The angular velocity gain to rotate the robot towards the goal
54     Kp_beta : The offset angular velocity gain accounting for smooth merging to
55              the goal angle (i.e., it helps the robot heading to be parallel
56              to the target angle.)
57     """
58
59     def __init__(self, Kp_rho, Kp_alpha, Kp_beta):
60         self.Kp_rho = Kp_rho
61         self.Kp_alpha = Kp_alpha
62         self.Kp_beta = Kp_beta
63

```

```

64 def calc_control_command(self, x, x_goal, theta, theta_goal):
65     """
66     Returns the control command for the linear and angular velocities as
67     well as the distance to goal
68
69     Parameters
70     -----
71     x : The current position in 2D
72     x_goal : The target position in 2D
73     theta : The current heading angle of robot with respect to x axis
74     theta_goal: The target angle of robot with respect to x axis
75
76     Returns
77     -----
78     rho : The distance between the robot and the goal position
79     v : Command linear velocity
80     w : Command angular velocity
81     """
82
83     # Description of local variables:
84     # - alpha is the angle to the goal relative to the heading of the robot
85     # - beta is the angle between the robot's position and the goal
86     #   position plus the goal angle
87     # - Kp_rho*rho and Kp_alpha*alpha drive the robot along a line towards
88     #   the goal
89     # - Kp_beta*beta rotates the line so that it is parallel to the goal
90     #   angle
91     #
92     # Note:
93     # we restrict alpha and beta (angle differences) to the range
94     # [-pi, pi] to prevent unstable behavior e.g. difference going
95     # from 0 rad to 2*pi rad with slight turn
96
97     # Proportional control , PI , PID
98     rho = np.hypot(x_diff, y_diff)
99     x_diff = x_goal - x # x.shape = 2 aka np.array
100    dhat = np.array([np.cos(theta), np.sin(theta)])
101    x_err = (x_diff @ dhat) ← scalar
102
103    moving_angle = np.arctan2(x_diff[1], x_diff[0])
104    moving_angle_err = Angle.diff(np.asarray(moving_angle),
105                                  np.asarray(theta))
106
107    dest_angle_err = Angle.diff(np.asarray(theta_goal),
108                                np.asarray(theta))
109
110    v = self.Kp_rho * x_err
111    w = (self.Kp_alpha * moving_angle_err
112         if (np.linalg.norm(x_diff) > 0.001) else
113         controller.Kp_beta * dest_angle_err)
114    return np.array([v, w])
115
116 def control(self, state, state_goal):
117     return self.calc_control_command(state[:2], state_goal[:2], state[2],
118                                       state_goal[2])
119
120 def rotmat2D(theta):
121     return np.vstack([np.hstack([np.cos(theta), -np.sin(theta)]),
122                       np.hstack([np.sin(theta), np.cos(theta)])])
123
124 def move_to_pose(controller,
125                   x_start, y_start, theta_start, x_goal, y_goal, theta_goal,
126                   dt = 0.01,
127                   # Robot specifications
128                   MAX_LINEAR_SPEED = 15,
129                   MAX_ANGULAR_SPEED = 7,
130                   show_animation = True)

```



```

129         ):
130
131     pos_goal = np.array([x_goal, y_goal])
132     x = np.array([x_start, y_start])
133     theta = theta_start
134
135     x_diff = pos_goal - x
136
137     pos_traj = []
138
139     rho = np.linalg.norm(x_diff)
140     while rho > 0.001 and np.abs(Angle.diff(np.asarray(theta_goal),
141                                           np.asarray(theta))) > 0.001:
142         pos_traj.append(x)
143
144         u = controller.calc_control_command(
145             x, pos_goal, theta, theta_goal)
146         v = u[0]
147         w = u[1]
148
149         if abs(v) > MAX_LINEAR_SPEED:
150             v = np.sign(v) * MAX_LINEAR_SPEED
151
152         if abs(w) > MAX_ANGULAR_SPEED:
153             w = np.sign(w) * MAX_ANGULAR_SPEED
154
155         theta = Angle.wrap(theta + w * dt)
156         x = x + v * np.array([np.cos(theta), np.sin(theta)]) * dt
157         x_diff = pos_goal - x
158         rho = np.linalg.norm(x_diff)
159
160         if show_animation: # pragma: no cover
161             plt.cla()
162             plt.arrow(x_start, y_start, np.cos(theta_start),
163                      np.sin(theta_start), color='r', width=0.1)
164             plt.arrow(x_goal, y_goal, np.cos(theta_goal),
165                      np.sin(theta_goal), color='g', width=0.1)
166             plot_vehicle(x[0], x[1], theta,
167                         [p[0] for p in pos_traj],
168                         [p[1] for p in pos_traj],
169                         dt=dt)
170
171
172 def plot_vehicle(x, y, theta, x_traj, y_traj, dt): # pragma: no cover
173     # Corners of triangular vehicle when pointing to the right (0 radians)
174     p1_i = np.array([0.5, 0, 1]).T
175     p2_i = np.array([-0.5, 0.25, 1]).T
176     p3_i = np.array([-0.5, -0.25, 1]).T
177
178     T = transformation_matrix(x, y, theta)
179     p1 = np.matmul(T, p1_i)
180     p2 = np.matmul(T, p2_i)
181     p3 = np.matmul(T, p3_i)
182
183     plt.plot([p1[0], p2[0]], [p1[1], p2[1]], 'k-')
184     plt.plot([p2[0], p3[0]], [p2[1], p3[1]], 'k-')
185     plt.plot([p3[0], p1[0]], [p3[1], p1[1]], 'k-')
186
187     plt.plot(x_traj, y_traj, 'b--')
188
189     # for stopping simulation with the esc key.
190     plt.gcf().canvas.mpl_connect(
191         'key_release_event',
192         lambda event: [exit(0) if event.key == 'escape' else None])
193

```

```

194 plt.xlim(0, 20)
195 plt.ylim(0, 20)
196
197 plt.pause(dt)
198
199
200 def transformation_matrix(x, y, theta):
201     return np.array([
202         [np.cos(theta), -np.sin(theta), x],
203         [np.sin(theta), np.cos(theta), y],
204         [0, 0, 1]
205     ])
206
207 def unicycle_f(x_t, u_t, dt):
208     theta = x_t[2]
209     return np.array([x_t[0] + u_t[0] * np.cos(theta) * dt,
210                     x_t[1] + u_t[0] * np.sin(theta) * dt,
211                     x_t[2] + u_t[1] * dt])
212
213 def unicycle_Jf_x(x_t, u_t, dt):
214     theta = x_t[2]
215     return np.array([
216         [1., 0., -u_t[0] * np.sin(theta) * dt],
217         [0., 1., u_t[0] * np.cos(theta) * dt],
218         [0., 0., 1.]
219     ])
220
221 def unicycle_Jf_u(x_t, u_t, dt):
222     theta = x_t[2]
223     return np.array([
224         [np.cos(theta) * dt, 0.],
225         [np.sin(theta) * dt, 0.],
226         [0., dt]
227     ])
228
229
230
231 def main():
232     # simulation parameters
233     dt = 0.01
234     pid_controller = PIDController(9, 15, 3)
235     lqr_controller = iLQRController(
236         Q = np.diag([0.9, 0.9, 0.1]),
237         R = np.eye(2) * 0.01,
238         f = unicycle_f,
239         Jf_x = unicycle_Jf_x,
240         Jf_u = unicycle_Jf_u,
241         dt = dt,
242         init_controller = pid_controller)
243     controller = lqr_controller
244
245     for i in range(5):
246         x_start = 20 * random()
247         y_start = 20 * random()
248         theta_start = 2 * np.pi * random() - np.pi
249         x_goal = 20 * random()
250         y_goal = 20 * random()
251         theta_goal = 2 * np.pi * random() - np.pi
252         print("Initial x: %.2f m\nInitial y: %.2f m\nInitial theta: %.2f rad\n" %
253               (x_start, y_start, theta_start))
254         print("Goal x: %.2f m\nGoal y: %.2f m\nGoal theta: %.2f rad\n" %
255               (x_goal, y_goal, theta_goal))
256         move_to_pose(controller,
257                     x_start, y_start, theta_start, x_goal, y_goal,
258                     theta_goal,

```

```
259             dt = dt)
260
261
262 if __name__ == '__main__':
263     main()
```